Cairo University
Faculty of Engineering
Electronics & Communications Dept.

Neural Networks

Assignment #3

Synthetic Data Generation with VAE & GAN

Prepared by:

Name	ID
Zeyad Antar Othman Abu-Zaid	9220331
Abdelrahman Hatem Nasr Youssef	9220461
Abdallah Essam Mohamed Mahmoud	9220479

Submitted to:

Dr. Mohsen Rashwan

Dr. Mohamed Abdelghani

GitHub: GitHub Repository

May 2026

Contents

1 Introduction

Deep learning models need a lot of data to work well, but getting large labelled datasets is often expensive or just not possible. In this assignment we look at ways to boost model performance when training data is limited.

For **Problems 1 and 2** we focus on **synthetic data generation** using the ReducedMNIST dataset (10 digit classes, 28×28 images, only **350 training samples per class**). We train a **Conditional VAE (CVAE)** and a **Conditional DCGAN (cDCGAN)**, use them to generate new images, filter those images by how confident a classifier is about them, and then check whether adding them actually helps classification.

2 Problem 1 — Synthetic Data Generation with Conditional VAE

2.1 Methodology

2.1.1 Data Loading and Preparation

The **ReducedMNIST** dataset has folders `0/` through `9/`, each holding grayscale JPEG images of that digit. Although the full training set contains 1 000 images per class, we only use the **first 350 per class** on purpose to simulate having limited data. Every image is resized to 28×28 and normalised to $[0, 1]$.

2.1.2 Data Augmentation

With only 350 images per class there is a real risk of poor test accuracy. To help with this we apply **15× data augmentation** through simple geometric transforms:

- **Random rotation:** up to $\pm 15°$
- **Random translation:** up to 10% horizontally and vertically
- **Random scaling:** between 90% and 110% of original size

These transforms keep each digit recognisable while adding variety similar to real handwriting differences. The training set grows from $350 \times 10 = 3{,}500$ to roughly $5{,}600 \times 10 = 56{,}000$ samples.

2.1.3 CVAE Architecture

A VAE learns a smooth latent space that we can sample from to create new images. By giving it the class label as an extra input, we get to choose *which digit* gets generated.

- **Encoder:** The input image $(1, 28, 28)$ is concatenated with a learnable *label map* — a 28×28 embedding of the class label — along the channel dimension, yielding a $(2, 28, 28)$ tensor. Three convolutional layers (with BatchNorm and ReLU) downsample this to a 2 048-dimensional feature vector, which is projected to the mean $\boldsymbol{\mu}$ and log-variance $\log \boldsymbol{\sigma}^2$ of a 64-dimensional latent Gaussian.
- **Reparameterization:** A latent code $\mathbf{z}$ is sampled via $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.
- **Decoder:** The latent code $\mathbf{z}$ is concatenated with a separate label embedding and projected back to $(128, 4, 4)$. Three transposed-convolution layers upsample this to the reconstructed image $(1, 28, 28)$ with a final Sigmoid activation.

2.1.4 Loss Function

The CVAE is trained to minimise:

$$\mathcal{L}_{\text{CVAE}} = \underbrace{\text{BCE}(\hat{\mathbf{x}}, \mathbf{x})}_{\text{reconstruction}} + \underbrace{D_{\text{KL}}\big(\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2) \parallel \mathcal{N}(\mathbf{0}, \mathbf{I})\big)}_{\text{regularisation}}$$

The **BCE term** pushes the output to look like the input, while the **KL term** keeps the latent distribution close to a standard normal so that we can smoothly interpolate between digits and sample meaningful new ones.

2.1.5 LeNet-5 Classifier

We use LeNet-5 for two things:

1. **Confidence filtering:** we first train it on the 350 real samples and use its softmax scores to judge how good each generated image looks.
2. **Final evaluation:** we then retrain it from scratch on real + synthetic data and measure test accuracy.

Architecture: Conv(1→6, 5×5) → ReLU → MaxPool → Conv(6→16, 5×5) → ReLU → MaxPool → FC(400→120) → FC(120→84) → FC(84→10). Trained for 20 epochs with Adam (lr = 10^{-3}).

2.1.6 Generation and Filtering Pipeline

1. **Generate:** 5 independent runs × 1 000 samples per class = 50 000 total.

2. **Score:** the pre-trained LeNet-5 computes the maximum softmax probability (confidence) for each generated image.
3. **Filter into three datasets:**
   - **Dataset A:** all 50 000 generated samples (no filtering).
   - **Dataset B:** only samples with confidence ≥ 0.9 (high quality).
   - **Dataset C:** samples with $0.6 \leq$ confidence < 0.9 (medium quality).
4. **Retrain:** for each dataset, combine the 350 real samples with the synthetic subset and retrain LeNet-5 from scratch. Evaluate on the test set.

2.2 Experimental Setup

Parameter	**Value**
Dataset	ReducedMNIST (28×28, 10 classes)
Real training data	350 images/class (limited)
Test data	200 images/class (2 000 total)
Augmentation	15× (rotation, translation, scaling)
CVAE latent dim	64
CVAE epochs	50
CVAE batch size	128
CVAE optimiser	Adam, lr = 10^{-3}
LeNet-5 epochs	20
LeNet-5 optimiser	Adam, lr = 10^{-3}
LeNet-5 batch size	64
Generation	5 runs × 1 000/class = 50 000
Confidence filter	LeNet-5 max-softmax

Table 1. Problem 1 hyperparameters (CVAE).

2.3 Results

Training Data	Synthetic Samples	Test Accuracy (%)
350 real (baseline)	0	95.90
350 real + VAE Dataset A (all)	50 000	97.30
350 real + VAE Dataset B (≥0.9)	32 968	96.85
350 real + VAE Dataset C (0.6–0.9)	11 821	98.45
1 000 real (upper baseline)	0	97.60

Table 2. LeNet-5 accuracy when augmented with CVAE-generated samples.

2.4 Analysis

Every synthetic dataset beats the 350-sample baseline of 95.90%. **Dataset C** (medium confidence, $0.6 \leq \text{conf} < 0.9$) does the best at **98.45%**, which is even higher than the 1 000-real upper baseline (97.60%).

Dataset B (≥ 0.9 confidence) only reaches 96.85%. It seems like the highest-confidence VAE samples are "too safe" — they look a lot like what the model already saw during training, so they do not really teach it anything new.

Dataset C has more variety: slightly unusual stroke styles, borderline shapes, and so on. That extra diversity helps the classifier generalise, almost like a form of regularisation.

The main takeaway here is that medium-confidence filtering works better than both no filtering and strict high-confidence filtering, at least for VAE-generated data.

3 Problem 2 — Synthetic Data Generation with Conditional DCGAN

3.1 Methodology

Everything from data loading to augmentation to the LeNet-5 pipeline stays the same as in Problem 1. The only difference is that we swap the CVAE for a **Conditional Deep Convolutional GAN (cDCGAN)**.

3.1.1 Generator Architecture

1. A 100-dimensional noise vector $\mathbf{z}$ is concatenated with a 10-dimensional label embedding.
2. A fully connected layer projects the 110-dimensional input to $256{\times}7{\times}7$ (with BatchNorm + ReLU).
3. Two transposed convolution layers upsample to $128 \times 14 \times 14$ and then $64 \times 28 \times 28$.
4. A final 3×3 convolution with Sigmoid produces the $(1, 28, 28)$ output.

3.1.2 Discriminator Architecture

1. The input image $(1, 28, 28)$ is concatenated with a label map $(1, 28, 28)$ along the channel dimension.
2. Two convolutional layers (with LeakyReLU and Dropout) downsample to $128 \times 7 \times 7$.
3. A fully connected layer outputs a single real/fake probability (Sigmoid).

3.1.3 Training Details

- **Loss:** Binary Cross-Entropy (BCE).
- **Label smoothing:** Real targets set to 0.9 instead of 1.0 to stabilise training.
- **Optimiser:** Adam with lr = 2×10^{-4}, $\beta_1 = 0.5$, $\beta_2 = 0.999$ (standard DCGAN hyperparameters).
- **Epochs:** 100.

3.2 Experimental Setup

Parameter	Value
cDCGAN noise dim	100
cDCGAN epochs	100
cDCGAN batch size	128
cDCGAN optimiser	Adam, lr = 2×10^{-4}, $\beta = (0.5, 0.999)$
Label smoothing	0.9
Generation	5 runs × 1 000/class = 50 000
Confidence filter	LeNet-5 max-softmax

Table 3. Problem 2 hyperparameters (cDCGAN). Data, augmentation, and LeNet-5 settings are the same as Table 1.

3.3 Results

Training Data	Synthetic Samples	Test Accuracy (%)
350 real (baseline)	0	95.90
350 real + GAN Dataset A (all)	50 000	97.45
350 real + GAN Dataset B (≥0.9)	35 512	97.45
350 real + GAN Dataset C (0.6–0.9)	10 120	97.55
1 000 real (upper baseline)	0	97.25

Table 4. LeNet-5 accuracy when augmented with cDCGAN-generated samples.

3.4 Analysis

All three GAN-augmented setups beat the 350-sample baseline. What stands out is how close the numbers are: Datasets A, B, and C give 97.45%, 97.45%, and 97.55%. The filtering strategy barely matters here, which tells us the cDCGAN is producing pretty good samples across the board.

Another thing worth noting is that the GAN ended up with a bigger Dataset B (35 512 vs. 32 968

for the VAE). GAN outputs tend to be *sharper*, so the classifier is more confident about them on average.

3.5 VAE vs. GAN Comparison

Configuration	VAE Acc. (%)	VAE Samples	GAN Acc. (%)	GAN Samples
350 real baseline	95.90	—	95.90	—
+ Dataset A (all generated)	97.30	50 000	97.45	50 000
+ Dataset B (conf ≥ 0.9)	96.85	32 968	97.45	35 512
+ Dataset C ($0.6 \leq$ conf < 0.9)	98.45	11 821	97.55	10 120
1 000 real baseline	97.60	—	97.25	—

Table 5. Side-by-side comparison of VAE vs. GAN synthetic augmentation.

- **VAEs** are trained with a reconstruction + KL loss, which gives them a smooth latent space. The downside is that their outputs tend to be a bit *blurry*. Still, the diversity in Dataset C turned out to be very helpful, giving us the best overall accuracy (**98.45%**).
- **GANs** learn by competing (generator vs. discriminator) and produce *sharper* images. Their numbers are steady across all filtering levels (97.45–97.55%), so they are a safe bet if you do not want to worry about picking the right confidence threshold.
- **Best overall:** VAE Dataset C at 98.45%. The slightly blurry but diverse samples from the CVAE end up being the most useful for training.
- **Both models beat the 350-sample baseline** and get close to (or pass) the 1 000-real upper bound, which shows that generating synthetic data really does work when you do not have enough real images.

4 Comprehensive Summary and Comparison with Assignment 2

4.1 Which GAN Selection Strategy Gives the Best Improvement?

Out of the three filtering strategies, **Dataset C** (medium confidence, $0.6 \leq$ conf < 0.9) gives the best GAN accuracy at **97.55%**, just slightly above Datasets A and B (both 97.45%). For the CVAE the gap is bigger — Dataset C hits **98.45%**, the best number we saw anywhere.

Why does medium confidence win? Those samples are different enough to teach the classifier something new, but still look enough like real digits to be useful. The high-confidence ones (Dataset B) are basically copies of what the model already knows, and the unfiltered set (Dataset A) has some bad samples mixed in.

4.2 Does Synthetic Data Reduce the Need for Real Data?

Yes. Starting from just **350 real images per class**, adding either VAE or GAN samples pushes accuracy past the 350-real baseline (95.90%) and even past the **1 000-real upper baseline** (97.25–97.60%). VAE Dataset C gets to 98.45%, which we probably could not reach even with 1 000 real images. So if collecting more real data is hard or expensive, generating synthetic samples and filtering them carefully is a solid alternative.

4.3 Comparison with Traditional Augmentation (Assignment 2)

In Assignment 2 we tested traditional augmentation (rotation, translation, Gaussian noise) and also plain GAN synthesis on the same ReducedMNIST dataset with LeNet-5. The table below puts all the important numbers from both assignments side by side.

Source	Configuration (350 real/class base)	Test Acc. (%)
Baselines		
Asgn. 2	350 real only	94.90–96.35
Asgn. 3	350 real only	95.90
Asgn. 2	1 000 real only	96.90–97.30
Asgn. 3	1 000 real only	97.25–97.60
Assignment 2 — Traditional Augmentation (Problem 5)		
Asgn. 2	350 real + 1 000 augmented/digit	96.80
Asgn. 2	350 real + 2 000 augmented/digit	97.20
Assignment 2 — GAN Synthesis (Problem 6, no filtering)		
Asgn. 2	350 real + 1 000 GAN synthetic/digit	95.80
Asgn. 2	350 real + 1 500 GAN synthetic/digit	96.60
Assignment 2 — Combined Aug. + GAN (Problem 6.3)		
Asgn. 2	350 real + 1 000 aug + 1 000 syn	**97.75**
Assignment 3 — CVAE with Confidence Filtering (Problem 1)		
Asgn. 3	350 real + VAE Dataset A (all, 50 000)	97.30
Asgn. 3	350 real + VAE Dataset B (≥0.9, 32 968)	96.85
Asgn. 3	350 real + VAE Dataset C (0.6–0.9, 11 821)	**98.45**
Assignment 3 — cDCGAN with Confidence Filtering (Problem 2)		
Asgn. 3	350 real + GAN Dataset A (all, 50 000)	97.45
Asgn. 3	350 real + GAN Dataset B (≥0.9, 35 512)	97.45
Asgn. 3	350 real + GAN Dataset C (0.6–0.9, 10 120)	97.55

Table 6. Consolidated results across Assignments 2 and 3 (all using 350 real images/class as the base training set).

4.4 Key Takeaways

1. **Filtering makes a big difference.** In Assignment 2 we just threw all GAN samples into training and got modest gains (95.80–96.60%). Here in Assignment 3, using confidence-based filtering on the same type of GAN bumps accuracy to 97.45–97.55%. The lesson is clear: it is not just about generating more data, it is about picking the *right* generated data.
2. **VAE Dataset C wins overall** at 98.45%, beating everything else including Assignment 2's best combined setup (97.75%). The CVAE's smooth latent space seems to produce just the right mix of variety and quality in the medium-confidence range.
3. **Plain augmentation still works really well.** Back in Assignment 2, simply rotating and shifting 350 real images gave 97.75% — no generative model needed. It is free, simple, and hard to beat, so it should always be the first thing to try.
4. **We can get by with less real data.** Several setups using only 350 real images per class beat the 1 000-real baseline. That means generative augmentation can cut the real data requirement by over 65% without hurting accuracy.

5 Problem 3 — Understanding the Impact of Attention Mechanisms

5.1 Objective

The goal is to compare a **baseline CNN** (LeNet-5) against the same network augmented with a **spatial attention module** on two tasks:

1. **ReducedMNIST** — the same digit-image dataset from Problems 1 and 2.
2. **Spoken Digits** — classifying Mel-spectrogram images of spoken digits (the audio dataset from Assignment 2).

We measure *accuracy* and *training time* for each configuration.

5.2 Methodology

5.2.1 What Is Spatial Attention?

A plain CNN applies the same convolution everywhere equally. **Spatial attention** learns a 2-D mask (one value per spatial location) that highlights the most informative regions — for example, the stroke of a digit or a strong harmonic in a spectrogram — while fading out background clutter.

1. **Channel pooling:** compute the average and max across the channel dimension, producing two single-channel maps.
2. **Concatenation:** stack the two maps into a 2-channel tensor.
3. 3×3 **convolution → 1-channel mask.**
4. **Sigmoid:** normalise the mask to $[0, 1]$.
5. **Element-wise multiply:** apply the mask to the original feature maps.

The attention block is inserted **after the first pooling layer**, where the spatial resolution is still large enough for the 3×3 kernel to capture local saliency without blurring the whole map.

5.2.2 Network Architectures

ReducedMNIST — LeNet-5. Conv($1 \to 6$, 5×5) → ReLU → MaxPool → *[Spatial Attention]* → Conv($6 \to 16$, 5×5) → ReLU → MaxPool → FC(400→120) → FC(120→84) → FC(84→10).

Spoken Digits — SpectrogramCNN (from Assignment 2 Q3). For the spectrogram task we reuse the **exact 3-layer CNN from Assignment 2 Q3** (Conv–BN–ReLU–Pool ×3, with Dropout in the FC layers). This is important because the assignment explicitly asks to *revisit the spoken digits task from Assignment 2*, so LeNet-5 would not be the right baseline. Spatial attention is inserted after the first pool layer (16×16 maps).

5.3 Experimental Setup

Parameter	Value
Image size	32×32
Batch size	64
Epochs	15
Learning rate	10^{-3} (Adam)
Loss function	CrossEntropyLoss
Random seed	42
MNIST data	10 000 train / 2 000 test
Audio data	1 200 train / 300 test
Spectrogram	64-bin Mel, resized to 32×32

Table 7. Problem 3 hyperparameters.

5.4 Results

5.4.1 Core Comparison: Baseline vs. Spatial Attention

Task	Model	Accuracy (%)	Time (s)
ReducedMNIST	LeNet-5 (Baseline)	97.75	105.8
ReducedMNIST	LeNet-5 + Spatial Attention	**98.20**	113.8
Spoken Digits	SpectrogramCNN (A2 Baseline)	**97.67**	94.1
Spoken Digits	SpectrogramCNN + Spatial Attn	95.00	75.8

Table 8. Baseline vs. spatial attention on both tasks.

On ReducedMNIST, spatial attention gives a modest boost (+0.45 pp) with an 8% increase in training time. On spoken digits, however, the attention model *drops* by 2.67 pp compared to the strong SpectrogramCNN baseline. The extra parameters are not helping when the baseline is already very strong at 97.67%.

5.4.2 Bonus Experiments: Alternative Attention Mechanisms

We also implemented and tested four additional architectures to go beyond the required spatial attention comparison.

Task	Model	Attention	Acc. (%)	Time (s)
ReducedMNIST				
	CNN (Baseline)	None	97.75	105.8
	CNN + Spatial Attn	Spatial	98.20	113.8
	CNN + SE Block	Channel	97.85	112.5
	CNN + Self-Attention	Self	95.90	141.8
	ResNet-18 + SE	Channel	**98.40**	204.7
Spoken Digits (Spectrogram)				
	CNN (Baseline)	None	**97.67**	94.1
	CNN + Spatial Attn	Spatial	95.00	75.8
	CNN + SE Block	Channel	85.00	65.8
	CNN + Self-Attention	Self	64.00	70.4
	CNN + SpecAugment	None	72.33	67.3
	CNN + SE + SpecAug	Channel	61.67	82.7
	ResNet-18 + SE	Channel	**97.67**	65.8

Table 9. Master results table — all attention variants compared.

5.5 Analysis

1. **Model architecture matters more than attention.** Switching from LeNet-5 to SpectrogramCNN or ResNet-18 has a far bigger impact than adding any attention module. ResNet-18 + SE matches the SpectrogramCNN baseline at 97.67% on spoken digits while also topping the MNIST leaderboard at 98.40%.
2. **Spatial attention helps on MNIST but hurts on spectrograms.** On clean, centred digit

images the attention mask learns to focus on the stroke. On spectrograms, the discriminative information is spread across *frequency bands* (channels), not spatial locations — so spatial attention is the wrong inductive bias.

3. **Self-attention is data-hungry.** The Transformer-style head (95.90% on MNIST, 64.00% on audio) underfits on these small datasets. It needs more data or aggressive augmentation to shine.
4. **SE (channel) attention on a shallow backbone underperforms.** LeNet-5 only has 6 and 16 channels, so there is not much for the SE block to re-weight. On ResNet-18 (64–512 channels), channel attention becomes effective again.
5. **Training-time overhead is small.** Spatial attention adds about 7% wall-clock time on LeNet-5. The self-attention head is the most expensive at $+34\%$ due to the matrix multiplications in the Transformer layer.

5.6 Comparison with Assignment 2

In Assignment 2, the SpectrogramCNN baseline for spoken digits achieved ~99% accuracy on a clean train/test split. Here we see 97.67% instead, which is slightly lower because we use a smaller data subset and a fixed seed. The key takeaway remains the same: **model architecture** (deeper CNNs with BatchNorm and Dropout) matters far more than bolting on attention when the task is already well-suited to convolutions.

6 Problem 4 — General Information Retrieval (Arabic Q/A)

6.1 Objective

Build a complete **Arabic question-answering pipeline** that demonstrates three levels of retrieval:

1. **Classical search** (TF-IDF).
2. **Semantic search** (dense embeddings + FAISS).
3. **Retrieval-Augmented Generation (RAG)** — a small LLM answers using retrieved context.

We compare all three against a baseline of **LLM-only generation** (no retrieval) over 10 Arabic questions about a selected book.

6.2 Methodology

6.2.1 Selected Book

Attribute	Details
Title	Muqaddimah (مقدمة ابن خلدون)
Author	Abd al-Rahman ibn Khaldun
Source	Internet Archive (public domain)
Raw size	2 037 175 characters (two volumes combined)
Clean size	1 717 070 Arabic characters

Table 10. Source text details.

We chose the Muqaddimah because it is less commonly used in NLP benchmarks than the Quran, and its rich vocabulary in sociology, history, and philosophy makes retrieval genuinely useful — the LLM is less likely to have memorised every paragraph.

6.2.2 Text Preprocessing and Chunking

1. **Sentence splitting** — split on Arabic sentence terminators (. ! ؟ and newlines).
2. **Filtering** — discard fragments shorter than 10 characters (page numbers, headers).
3. **Chunking** — group 2–4 consecutive sentences into one passage, yielding **10 946 chunks**

from **32 680 sentences**.

4. **Embedding prefix** — each chunk is prefixed with `passage:` as required by the E5 model family.

6.2.3 Embedding and Indexing

Component	Details
Embedding model	`intfloat/multilingual-e5-small` (384-dim)
Encoding time	~23 s on CUDA (RTX 2050)
Vector DB	FAISS `IndexFlatIP` (exact cosine search)
Classical index	`TfidfVectorizer`, 50 000-feature sparse matrix
Passage count	10 946 chunks

Table 11. Retrieval infrastructure.

6.2.4 LLM for Generation

Attribute	Details
Model	`Qwen/Qwen2.5-1.5B-Instruct`
Size	~1.5 B parameters
Precision	float16 (offloaded to CPU due to VRAM constraints)
Generation	`max_new_tokens` = 512, temp = 0.7, top-p = 0.91
Avg. time/query	~177 s (RAG + LLM-only combined)

Table 12. LLM configuration.

6.2.5 Generation Modes

- **RAG:** Top-5 semantically retrieved chunks are injected into the prompt as context. The model is instructed to answer *only* from these passages.
- **LLM-only:** The same model answers from its internal parametric knowledge only — no retrieval.

6.3 Query Design

We defined 10 Arabic questions covering direct, indirect, and comparative question types at varying difficulty levels:

#	Query (translated)	Type	Difficulty
1	Divisions of knowledge	Direct	Easy
2	Definition of human civilisation	Direct	Medium
3	Climate's effect on peoples' morals	Indirect	Hard
4	Bedouins vs. sedentary people	Indirect	Medium
5	Organic vs. military kingship	Comparative	Hard
6	Theory of state formation	Conceptual	Hard
7	Who is Ibn Khaldun?	Factual	Easy
8	What is the Muqaddimah?	Factual	Easy
9	Role of *asabiyyah* in state formation	Indirect	Medium
10	Five stages of the state	Factual	Medium

Table 13. The 10 Arabic queries used for evaluation.

6.4 Results

6.4.1 Classical vs. Semantic Retrieval Scores

Query #	TF-IDF Top-1 Score	Semantic Top-1 Score
Q1	0.2310	0.8923
Q2	0.2610	0.8787
Q3	0.1766	0.8516
Q4	0.2520	0.8826
Q5	0.2984	0.8615
Q6	0.3170	0.9127
Q7	0.3673	0.8880
Q8	0.4723	0.8771
Q9	0.2249	0.8965
Q10	0.2236	0.8693
Mean	**0.2824**	**0.8810**

Table 14. Top-1 retrieval scores: TF-IDF vs. semantic (cosine similarity).

Semantic search consistently outperforms TF-IDF by a wide margin. The mean semantic top-1 score (**0.8810**) is more than 3× the mean TF-IDF score (**0.2824**). The gap is especially large for indirect and conceptual queries (Q3, Q9, Q10) where the answer uses different surface words than the question.

6.4.2 RAG vs. LLM-Only Generation

Q#	Query (abridged)	Gen. Time (s)
1	Divisions of knowledge	186.1
2	Definition of civilisation	219.6
3	Climate and morals	243.5
4	Bedouins vs. sedentary	145.5
5	Organic vs. military kingship	166.0
6	State formation	207.6
7	Who is Ibn Khaldun?	108.6
8	What is the Muqaddimah?	151.9
9	Role of *asabiyyah*	152.5
10	Five stages of the state	185.2
Mean		**176.7**

Table 15. Generation time per query pair (RAG + LLM-only combined).

6.5 Qualitative Analysis: RAG vs. LLM-Only

For each of the 10 questions, we compared the RAG answer (with top-5 retrieved context) against the LLM-only answer (no retrieval). Key observations:

1. **RAG answers are more grounded.** The RAG model cites specific concepts from the Muqaddimah (e.g., "*al-umran*" as the study of human society, the role of *asabiyyah* in state formation). The LLM-only model invents plausible-sounding but often *factually wrong* details — for example, listing "physics" as one of Ibn Khaldun's divisions of knowledge, or giving incorrect birth dates.
2. **LLM-only hallucinations are severe for a 1.5 B model.** Without retrieval, the model produced Chinese characters mid-sentence, attributed incorrect books to Ibn Khaldun (e.g., "al-Tabaqat"), and gave wildly wrong dates (965–1029 instead of 1332–1406). RAG eliminates most of these errors by anchoring the answer in real text.
3. **RAG struggles with hard comparative questions.** For Q5 (organic vs. military kingship) and Q10 (five stages of the state), the retrieved passages did not contain a clean, direct answer, so the RAG model's output was rambling and repetitive. This is a retrieval quality problem, not a generation problem.

4. **Both modes produce fluent Arabic.** The Qwen model handles Arabic well overall. The main quality difference is *factual accuracy*, not language fluency.

6.6 TF-IDF vs. Semantic Search: Detailed Comparison

- **TF-IDF** matches *exact surface words*. It does well when the query contains distinctive keywords (Q8, "المقدمة", scored 0.47), but fails when the answer uses synonyms or paraphrases.
- **Semantic search** matches *meaning*. Its top-1 scores are consistently ≥ 0.85, even for indirect questions like Q3 ("climate and morals") where TF-IDF scores only 0.18.
- The semantic retriever uses `multilingual-e5-small` (384-dimensional embeddings), which is fast to encode (~23 s for 10 946 passages on GPU) and produces normalised vectors suitable for exact inner-product search via FAISS.

6.7 Summary and Key Takeaways

1. **Semantic retrieval dominates classical retrieval for Arabic.** A mean top-1 score of 0.88 vs. 0.28 — the dense embeddings capture meaning far better than TF-IDF on morphologically rich Arabic.
2. **RAG dramatically reduces hallucination.** For a small 1.5 B-parameter model, retrieval is not optional — it is essential. Without it, the model fabricates facts, mixes languages, and gives wrong dates.
3. **Retrieval quality is the bottleneck.** When the FAISS index returns the right passages, the LLM generates good answers. When it does not (hard comparative questions), the LLM struggles regardless. Future work should focus on better chunking, hybrid retrieval (TF-IDF + dense), and re-ranking.
4. **Arabic NLP needs more attention.** OCR artefacts in the source text (the Internet Archive scan has character-level noise) hurt both retrieval and generation. Cleaner corpora and Arabic-native embedding models would help.